Software Requirements Specification

Unified Service Scheduler

Version	1.0
Date	25 October 2026
Status	Delivered build `service-scheduler` 0.2.0
Readers	The client (dealership and platform owners) and future development teams

1. Introduction

1.1 Purpose

This document states what the Unified Service Scheduler does, the rules it enforces, the quality it guarantees, and how to install and run it. The client can use it to check that the system does what was agreed. A future development team can use it to understand the system before changing it. The design history and the market research behind the decisions are in the separate Technical Document.

1.2 Product in one paragraph

Customers book a service appointment for their motorbike or car at a dealership. The system only offers a time when a **service bay of the right type** and a **technician with the right skill, on shift**, are both free for the **whole job**, and it makes double booking impossible even when many people press "Book" at the same moment. Dealership staff run the workshop day, answer advice requests, and manage bays, technicians and staff accounts. A platform manager onboards new dealerships.

1.3 Scope

In scope: customer booking of maintenance packages; advice requests (inspection on request and customisation) that an advisor turns into a bookable job; appointment lifecycle; workshop-day board; technician job list; management of bays, technicians and staff; multi-dealership onboarding; REST API with an OpenAPI contract; logging, metrics and health checks.

Out of scope for this version: section 12.

1.4 Definitions

Term	Meaning
Dealership	A workshop with its own address, time zone, opening hours, bays and staff. Every business record belongs to one dealership
Bay	A place where a vehicle is worked on. Types: motorbike stand, car lift, diagnostic bay (both vehicle types)
Skill	A technician qualification: general servicing, engine & drivetrain, electrical & diagnostics, brakes & suspension, customisation & performance

Term	Meaning
Service package	A fixed-price job with a checklist, duration, required skill and bay type
Advice request	A customer's description of a problem (inspection) or an idea (customisation). An advisor approves it with a duration, skill, bay type and estimate, or declines it
Slot	A possible start time on the dealership's grid (15, 30 or 60 minutes)
Capacity	The number of (bay, technician) pairs free for the whole job at a slot
D+1	The next day a dealership is open
Idempotency key	A client-chosen header value that makes a repeated booking request return the first result instead of booking again

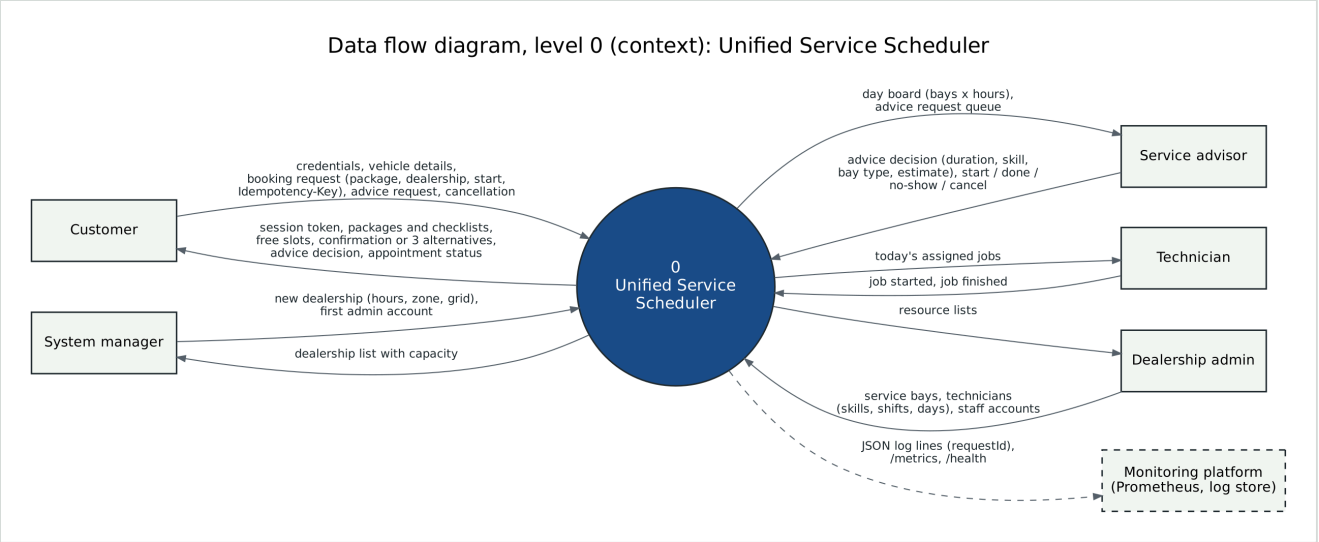
1.5 References

Document	Location
Technical Document	`01-Technical-Document.pdf`
Test Specification	`03-Test-Specification.pdf`
API contract (OpenAPI 3.1)	`backend/conf/openapi.yaml`, served at `/docs`
Database schema	`backend/conf/db/migration/V1__init.sql`
Diagram sources	`diagrams/` (PlantUML and Graphviz)
SOLID review, AI log	`docs/solid-review.md`, `docs/ai-log.md` in the source code

2. Overall description

2.1 Product perspective

A self-contained web application: a React single-page app talks JSON over HTTP to a Scala / Play API, which stores everything in PostgreSQL. It does not depend on any outside service at run time. It plays the role Keyloop's Service Booking and Service Hub play for UK dealerships, and goes further by scheduling bays and technicians rather than advisors only.



Data flow, level 0

2.2 User classes

Role	Who	Main goals
Customer	Vehicle owner, often with little technical knowledge	Book a trustworthy slot quickly; ask for advice when unsure what the vehicle needs
Service advisor	Front desk of one dealership	Run the workshop day; turn advice requests into schedulable jobs
Technician	Mechanic at one dealership	See today's jobs and record start and finish
Dealership admin	Manager of one dealership	Everything an advisor does, plus keep bays, technicians and staff accurate
System manager	Platform operator	Onboard dealerships and monitor their capacity

2.3 Operating environment

Component	Requirement
Server	Any OS with JDK 21 or newer (tested on 21 and 25). 1 vCPU and 1 GB RAM are enough for a demo
Database	PostgreSQL 15+ with the `btree_gist` extension. Development and tests use an embedded PostgreSQL 17 started by the API
Browser	Current Chrome, Edge, Firefox or Safari, desktop or phone
Development	sbt 1.10+, Node.js 22+

2.4 Constraints and assumptions

- A job needs exactly one bay and one technician for its whole duration; jobs do not span days.
- Start times sit on the dealership's slot grid inside opening hours; the job must end by closing.
- Bookings open at least 60 minutes ahead and up to 60 days ahead (both configurable per dealership).

- Bay types limit vehicle types: motorbike stand → motorbikes, car lift → cars, diagnostic bay → both.
- Customers book for themselves; staff manage bookings but do not create them in this version.
- Prices are in VND. All times are shown in the dealership's own time zone.

3. Functional requirements

Priority: **M** = must (core of the brief), **S** = should. Status: **Done** = built and covered by an automated test; **Done (UI)** = built and verified in the browser; **Partial** = built in part. Each requirement names its use case (UC) in the use case diagrams of section 9 and its manual test in the Test Specification.

3.1 Accounts and access

ID	Requirement	UC	Priority	Status	Test
FR-01	A visitor can register a customer account with name, email, optional phone and a password of at least 8 characters containing a letter and a digit	UC01	M	Done	TC-AUTH-03
FR-02	A user signs in with email and password and receives a bearer token valid for 12 hours (configurable). A failed sign-in does not reveal whether the email exists	UC02	M	Done	TC-AUTH-01, 02
FR-03	Signing out revokes the token on the server; it cannot be used again	UC02	M	Done	TC-AUTH-04
FR-04	Every API call is checked against the caller's role; staff only see and change data of their own dealership	—	M	Done	TC-AUTH-05, 06, TC-API-01, 02

3.2 Customer

ID	Requirement	UC	Priority	Status	Test
FR-10	A customer can add vehicles (type, make, model, plate, optional year, mileage, 17-character VIN) and list them. A plate is unique per owner	UC03	M	Partial (no edit or delete)	TC-CUS-01
FR-11	A customer can book a maintenance package in four steps: vehicle → package (only packages for that vehicle type) → dealership and time → confirm, with an optional note for the technician	UC04	M	Done	TC-CUS-03
FR-12	The time step shows, for each start time, whether it is free, how many places are left when only one is, and why it is not bookable (full, too soon, not open yet). It opens on the first day that still has a free time and refreshes every 30 seconds	UC20	M	Done	TC-CUS-03
FR-13	If the dealership has no bay or qualified technician for the job at all, the customer is told so and offered no times	UC20	M	Done	TC-CUS-05, TC-SYS-03
FR-14	If the chosen time is taken while the customer is confirming, the booking is refused and the three nearest free times are offered	UC22	M	Done	TC-CUS-04

ID	Requirement	UC	Priority	Status	Test
FR-15	A confirmed booking shows a reference (`SV-XXXXXX`), the technician and the dealership, and can be added to a calendar (.ics)	UC11	S	Done (UI)	TC-CUS-03
FR-16	A customer sees upcoming and past appointments and can cancel one until its start time	UC08, UC09	M	Done	TC-CUS-06
FR-17	A customer can send an advice request (inspection or customisation) for a vehicle at a dealership, choosing the areas concerned and describing it (10–2000 characters)	UC05, UC06	M	Done	TC-CUS-07, 08
FR-18	A customer can withdraw a request that is waiting or approved	UC10	S	Done	TC-CUS-08
FR-19	When an advisor approves a request, the customer sees the reply and the estimate and can pick a time; vehicle, job and dealership are fixed by the request	UC07	M	Done	TC-CUS-09, 10

3.3 Service advisor

ID	Requirement	UC	Priority	Status	Test
FR-30	An advisor sees one day of the dealership as a board (bays × hours) with every job, and can move between days	UC30	M	Done	TC-ADV-01
FR-31	Selecting a job shows its details (customer and phone, vehicle, bay, technician, price, note, checklist) and only the actions its status allows	UC30	M	Done	TC-ADV-02
FR-32	An advisor records that work started, that it is done, or that the customer did not come (only after the start time)	UC32	M	Done	TC-ADV-03
FR-33	An advisor can cancel a booking before work starts	UC33	M	Done	TC-ADV-04
FR-34	An advisor reviews advice requests of the dealership: approve with duration, skill, bay type (compatible with the vehicle), optional estimate and message; or decline with a reason of at least 5 characters	UC31	M	Done	TC-ADV-05, 06

3.4 Technician

ID	Requirement	UC	Priority	Status	Test
FR-40	A technician sees their own jobs for a day, with the checklist and the customer's note	UC40	M	Done	TC-TEC-01
FR-41	A technician starts and finishes their own jobs; they cannot move another technician's jobs	UC41	M	Done	TC-TEC-02

3.5 Dealership admin

ID	Requirement	UC	Priority	Status	Test
FR-50	An admin has every advisor permission	—	M	Done	TC-ADM-01
FR-51	An admin adds bays and takes them out of service or back; out-of-service bays are never offered	UC50	M	Done	TC-ADM-02
FR-52	An admin adds technicians with skills, shift times, working days and an optional login, and deactivates or reactivates them	UC51	M	Partial (no edit of an existing technician)	TC-ADM-03
FR-53	An admin creates staff accounts (service advisor or admin) for their dealership	UC52	S	Partial (no deactivate or password reset)	TC-ADM-04

3.6 System manager

ID	Requirement	UC	Priority	Status	Test
FR-60	A system manager onboards a dealership (code, name, address, IANA time zone, hours, open days) together with its first admin, in one step	UC60	M	Done	TC-SYS-02
FR-61	A system manager sees every dealership with its hours, active bays, active technicians and upcoming appointments (the API also returns the number of staff accounts)	UC61	S	Done	TC-SYS-01

4. Business rules

ID	Rule	Enforced in
BR-01	A bay, a technician and a vehicle can never have two live appointments (Confirmed or In the workshop) whose times overlap. Times are half-open, so 09:00–10:00 and 10:00–11:00 do not overlap	Three PostgreSQL `EXCLUDE USING gist` constraints; also checked in code before inserting
BR-02	A job is only offered when a bay of the required type and a technician with the required skill, on shift on that weekday, are free for the whole duration	`domain/Availability.scala`
BR-03	Best fit: among free technicians, the one with the fewest skills is chosen; ties go to the one with the least booked minutes that day. Specialists stay free for work only they can do	`Availability.allocate`
BR-04	A start time must be on an open day, on the slot grid, inside opening hours, at least `min_lead_minutes` ahead and at most `max_days_ahead` days ahead; the job must end by closing	`BookingService.validateStart`, the same rules in the slot grid

ID	Rule	Enforced in
BR-05	Lifecycle: Confirmed → In the workshop → Done; Confirmed → Cancelled (before work starts); Confirmed → Missed (no-show, only after the start). No way back from Done, Cancelled or Missed	`domain/Lifecycle.scala`
BR-06	A customer may only cancel; a technician may only start and finish their own jobs; advisors and admins may do all four moves at their dealership; a system manager may not move appointments	`Lifecycle.roleMayMoveTo`, `AppointmentService`
BR-07	An appointment is either for a package or for an approved advice request, never both. One advice request has at most one live booking	Database `CHECK` and partial unique index
BR-08	Cancelling the appointment of an advice request re-opens the request, so the customer can pick another time	`AppointmentService`
BR-09	An approved advice request must carry duration, skill and bay type, and the bay type must accept the vehicle type	Database `CHECK`; `ConsultationService.review`
BR-10	The same `Idempotency-Key` from the same customer never creates a second appointment; the repeat returns the first one	Unique index `(customer_id, idempotency_key)` and an advisory lock
BR-11	Every status change is recorded with who, when, from and to	`appointment_events`

5. Non-functional requirements

ID	Area	Requirement	How it is met and verified
NFR-01	Correctness under concurrency	Any number of simultaneous bookings for one slot produce exactly as many appointments as there is capacity, and never an error	`BookingConcurrencySpec` (50 at once); load test: 1000 at once → exactly 4 × 201 and 996 × 409, 0 errors
NFR-02	Throughput	One instance on 2 shared vCPUs answers a burst of 1000 simultaneous requests without errors	Load test: 1000 bookings spread over 54 slots in 3.2 s; 1000 availability reads in 2.1 s
NFR-03	Scalability	The API keeps no state in memory, so more instances can run behind a load balancer	Sessions and locks live in PostgreSQL; BR-01 holds across instances
NFR-04	Security	Passwords hashed with PBKDF2-HMAC-SHA512 and a per-user salt; bearer tokens stored only as SHA-256 hashes; tokens revocable; role and dealership checked on every call; no secrets in logs	`Passwords.scala`, `ApiSpec` role and isolation tests
NFR-05	Reliability	Deadlocks and lost races are retried (up to 5 attempts); transient database errors answer `503` with `Retry-After`	`BookingServiceFakesSpec`, error handler

ID	Area	Requirement	How it is met and verified
NFR-06	Observability	One JSON log line per event with request ID, user, role and dealership; Prometheus metrics; health check; slow-query log over 200 ms	`/metrics`, `/health`, TC-OPS-01 to 04
NFR-07	Traceability	Every response and error carries an `X-Request-Id`; the UI shows it for unexpected errors	`ApiSpec` › request-ID propagation
NFR-08	Maintainability	Hexagonal modules; inner layers never import adapters, HTTP, JDBC or Anorm; compiles with `-Wunused:all -Werror`	`ArchitectureSpec`
NFR-09	Testability	95 automated tests on the real database engine, including property-based tests over thousands of random workshops	`npm test`
NFR-10	Usability	Customer flow works on phones and desktops; availability opens on the first free day; plain-language errors	Verified in a headless browser at desktop and phone sizes
NFR-11	Time zones	Every dealership has its own IANA time zone; slot times stay correct across daylight-saving changes	`AvailabilitySpec` (Europe/London)
NFR-12	API contract	Every route is documented in OpenAPI 3.1	`ApiSpec` fails if a route is missing from `openapi.yaml`

6. External interfaces

6.1 User interface

Screen	Path	Roles
Sign in, Create account	`/login`, `/register`	Anyone
Home	`/`	Customer
Book a service	`/book`, `/book?request=<id>`	Customer
My appointments	`/appointments`	Customer
Advice requests, new request	`/requests`, `/requests/new?kind=INSPECTION|MODIFICATION`	Customer
My vehicles	`/vehicles`	Customer
Workshop day	`/staff/board`	Service advisor, admin
Advice requests (staff)	`/staff/requests`	Service advisor, admin
Bays & team	`/admin`	Admin
My jobs	`/jobs`	Technician

Screen	Path	Roles
Dealerships	`/system`	System manager

Screenshots of every screen are in the Test Specification.

6.2 REST API

JSON over HTTP. Authentication: `Authorization: Bearer <token>` from `POST /api/auth/login`. Errors always have the shape `{"error":{"code","message","details?","requestId"}}`. Times are ISO-8601 with offset. The full contract with request and response schemas is served at `/docs`.

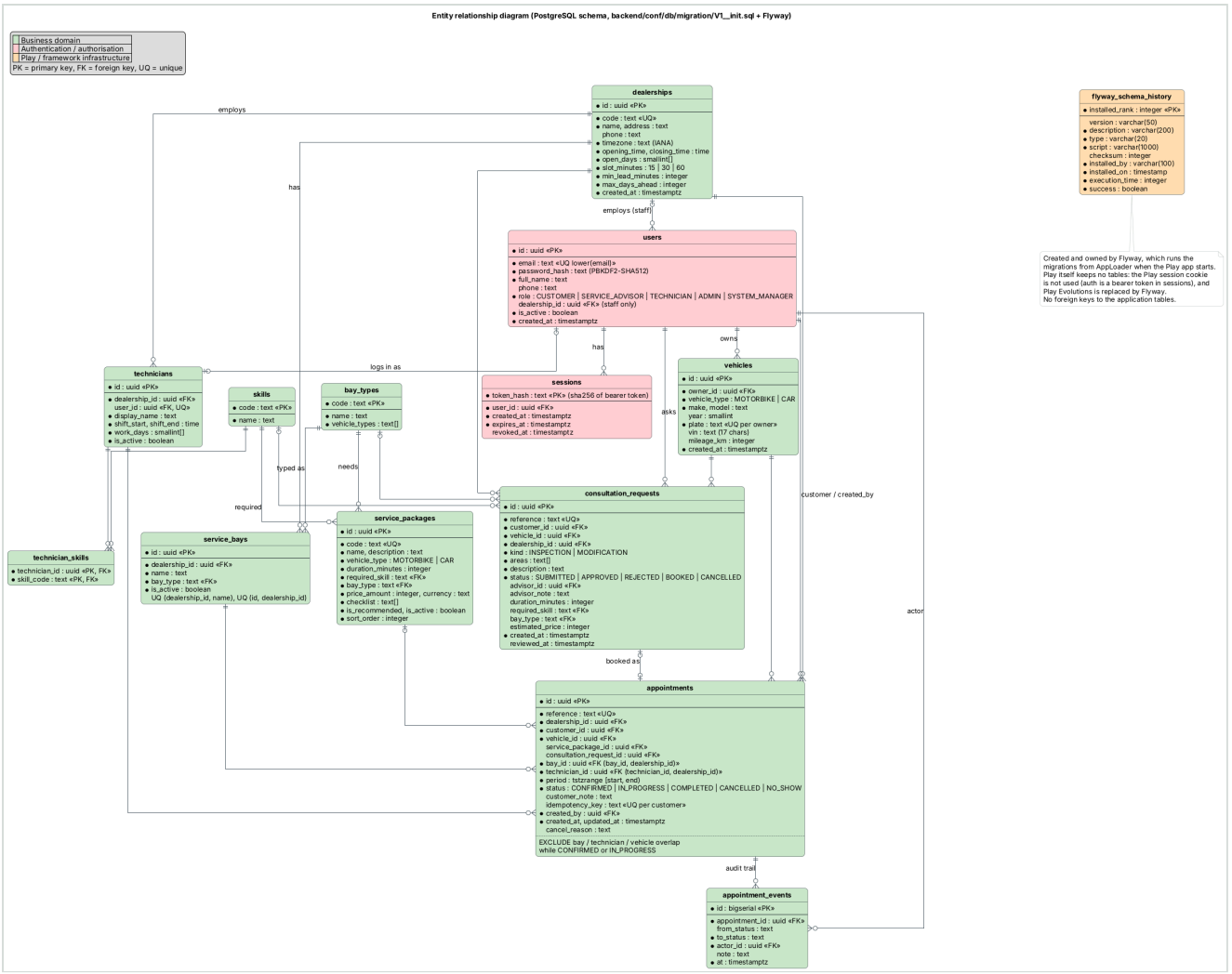
Method	Path	Purpose	Roles
POST	`/api/auth/register`	Create a customer account and sign in	Anyone
POST	`/api/auth/login`	Sign in	Anyone
POST	`/api/auth/logout`	Revoke the token	Signed in
GET	`/api/auth/me`	The signed-in user	Signed in
GET	`/api/dealerships`	Dealerships to book with	Signed in
GET	`/api/service-packages`	Maintenance packages	Signed in
GET	`/api/reference-data`	Inspection areas, skills, bay types	Signed in
GET, POST	`/api/me/vehicles`	List or add own vehicles	Customer
GET	`/api/availability`	Slot grid for a dealership, date and package or approved request	Customer, advisor, admin
POST	`/api/appointments`	Book (optional `Idempotency-Key`)	Customer
GET	`/api/me/appointments`	Own appointments	Customer
GET	`/api/appointments/:id`	One appointment	Owner, staff of its dealership
POST	`/api/appointments/:id/cancel`	Cancel	Customer (own, before start), advisor, admin
POST	`/api/appointments/:id/status`	Start, finish, no-show	Advisor, admin, technician (own)
GET	`/api/staff/appointments`	One day at the dealership	Advisor, admin
GET	`/api/staff/resources`	Active bays and technicians	Advisor, admin
GET	`/api/technician/jobs`	Own jobs for a day	Technician
POST	`/api/consultations`	Send an advice request	Customer
GET	`/api/me/consultations`	Own advice requests	Customer
GET	`/api/consultations/:id`	One advice request	Owner, staff of its dealership
POST	`/api/consultations/:id/cancel`	Withdraw	Customer (own)

Method	Path	Purpose	Roles
GET	`/api/staff/consultations`	The dealership's advice requests	Advisor, admin
POST	`/api/consultations/:id/review`	Approve or decline	Advisor, admin
GET, POST	`/api/admin/bays`	List or add bays	Admin
PATCH	`/api/admin/bays/:id`	Put in or out of service	Admin
GET, POST	`/api/admin/technicians`	List or add technicians	Admin
PATCH	`/api/admin/technicians/:id`	Activate or deactivate	Admin
GET, POST	`/api/admin/staff`	List or add staff accounts	Admin
GET, POST	`/api/system/dealerships`	List or onboard dealerships	System manager
GET	`/health`	Database check	Anyone
GET	`/metrics`	Prometheus metrics	Operator
GET	`/docs`, `/docs/openapi.yaml`	Swagger UI and the contract	Anyone

Main status codes: `201` created, `200` idempotent replay, `401` not signed in, `403` wrong role or dealership, `404` not found or not yours, `409 SLOT_UNAVAILABLE` (with `details.alternatives`), `409 TOO_LATE_TO_CANCEL`, `422` validation (`NOT_ON_SLOT_GRID`, `TOO_SOON`, `NOT_OFFERED_HERE`, `PACKAGE_NOT_FOR_VEHICLE`, …), `503 TRY_AGAIN`.

7. Data model

The schema has 14 tables in three groups. The entity relationship diagram colours them: **green** for the business domain, **red** for authentication and authorisation, **orange** for framework infrastructure.



Entity relationship diagram

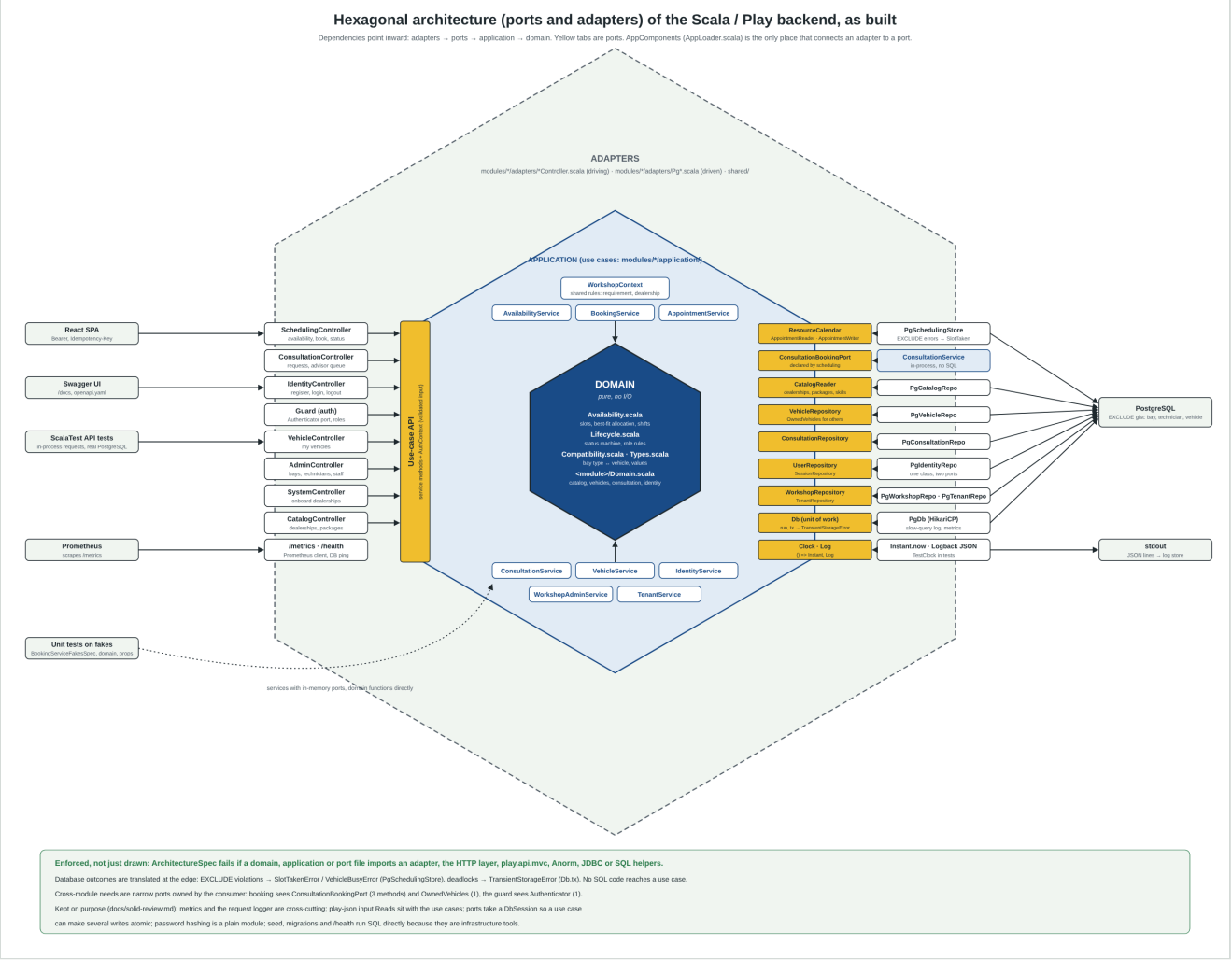
Group	Table	Holds
Business domain	`dealerships`	Tenants: code, address, time zone, hours, open days, slot size, lead time, booking horizon
Business domain	`skills`, `bay_types`	Reference data; a bay type lists the vehicle types it accepts
Business domain	`service_bays`	Bays per dealership, with type and in-service flag
Business domain	`technicians`, `technician_skills`	Technicians per dealership, shift, working days, skills, optional login
Business domain	`service_packages`	The catalogue: price, duration, skill, bay type, checklist
Business domain	`vehicles`	Customers' vehicles
Business domain	`consultation_requests`	Advice requests and the advisor's decision
Business domain	`appointments`	Bookings, with the period as `tstzrange` and the three exclusion constraints
Business domain	`appointment_events`	Audit trail of every status change
Authentication and authorisation	`users`	Every account, its role and (for staff) its dealership; password hash

Group	Table	Holds
Authentication and authorisation	`sessions`	SHA-256 hashes of bearer tokens, expiry, revocation
Framework infrastructure	`flyway_schema_history`	Applied migrations, written by Flyway when the Play application starts

Play Framework itself creates no tables: the application does not use Play's session cookie (it uses bearer tokens stored in `sessions`), and schema changes are made by Flyway instead of Play Evolutions. `flyway_schema_history` is therefore the only framework table.

8. System architecture

Hexagonal (ports and adapters) inside a modular monolith. Each module has the same layers: pure domain, application services that depend only on ports (traits), and adapters for HTTP and PostgreSQL. `AppLoader` is the only place that decides which adapter implements which port.



Hexagonal architecture

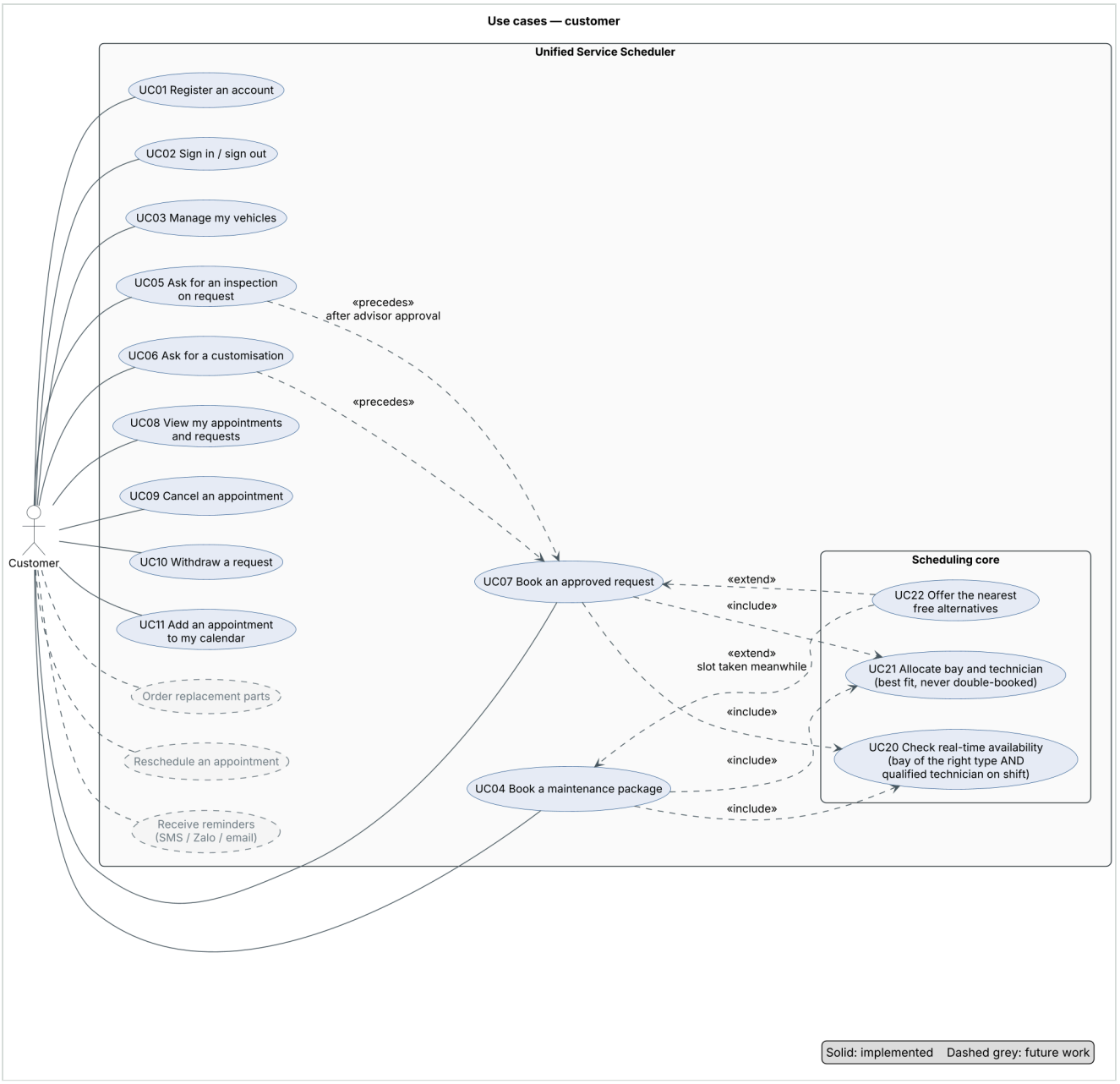
Module	Responsibility
`scheduling`	Availability, booking, appointment lifecycle

Module	Responsibility
`consultation`	Advice requests and their review
`vehicles`	Customers' vehicles
`catalog`	Dealerships, packages, reference data
`identity`	Accounts, passwords, sessions, the auth guard
`admin`	Bays, technicians and staff of a dealership
`system`	Dealership onboarding and monitoring
`shared`	Configuration, logging, errors, database port and pool, metrics, migrations
`http`	Request IDs, error format, request logging, `/health`, `/metrics`, `/docs`

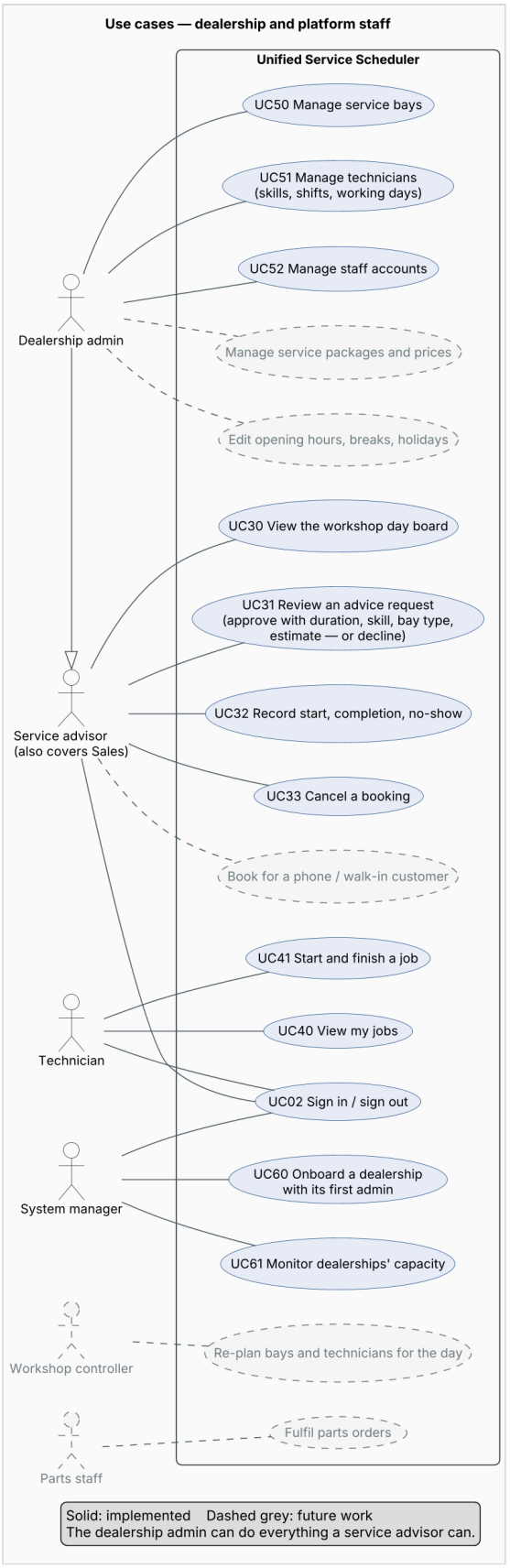
9. Diagrams

All diagrams are in the `diagrams` folder as PNG and SVG, with their PlantUML (`.puml`) or Graphviz (`.dot`) sources. Solid shapes are built; dashed grey shapes are future work.

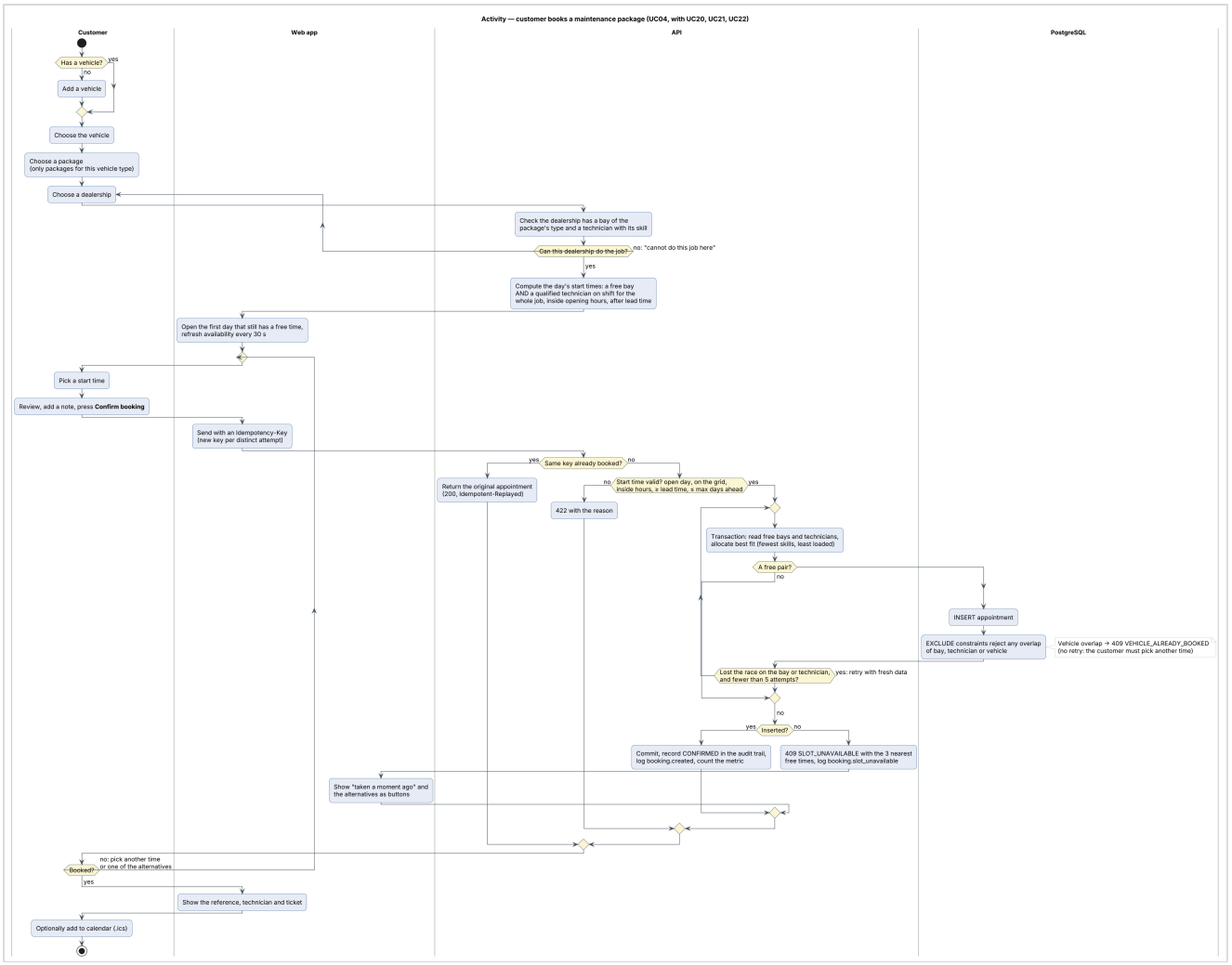
#	Diagram	File	Shows
1	Use cases, customer	`use-case-customer.png`	UC01–UC11, UC20–UC22
2	Use cases, staff	`use-case-staff.png`	UC30–UC61 for advisor, technician, admin, system manager
3	Activity, book a maintenance package	`activity-book-package.png`	The main customer flow with its checks
4	Activity, inspection or customisation	`activity-consult-first.png`	Consult first, then book
5	Activity, the workshop day	`activity-workshop-day.png`	Advisor and technician during the day
6	State, appointment lifecycle	`state-appointment.png`	Statuses and who may move between them
7	Data flow, level 0	`dfd-context.png`	The system and its external entities
8	Data flow, level 1	`dfd-level1.png`	Seven processes and eight data stores
9	Class, domain model	`class-domain-model.png`	Persisted entities with multiplicities
10	Class, application layer	`class-application.png`	Services, ports and their PostgreSQL adapters
11	Hexagonal architecture	`hexagonal-architecture.png`	Modules, ports and adapters as built
12	Entity relationship	`erd.png`	All 14 tables, colour-coded by group



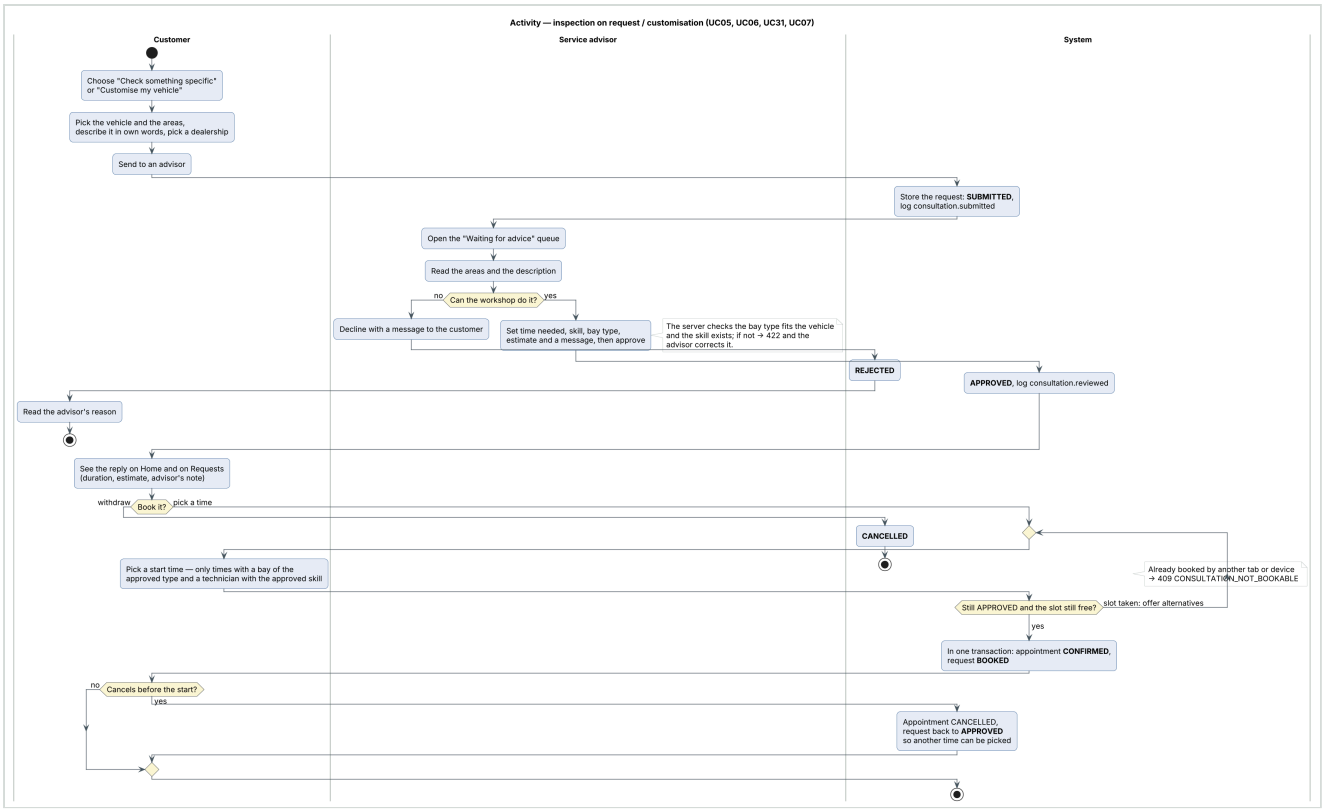
Use cases, customer



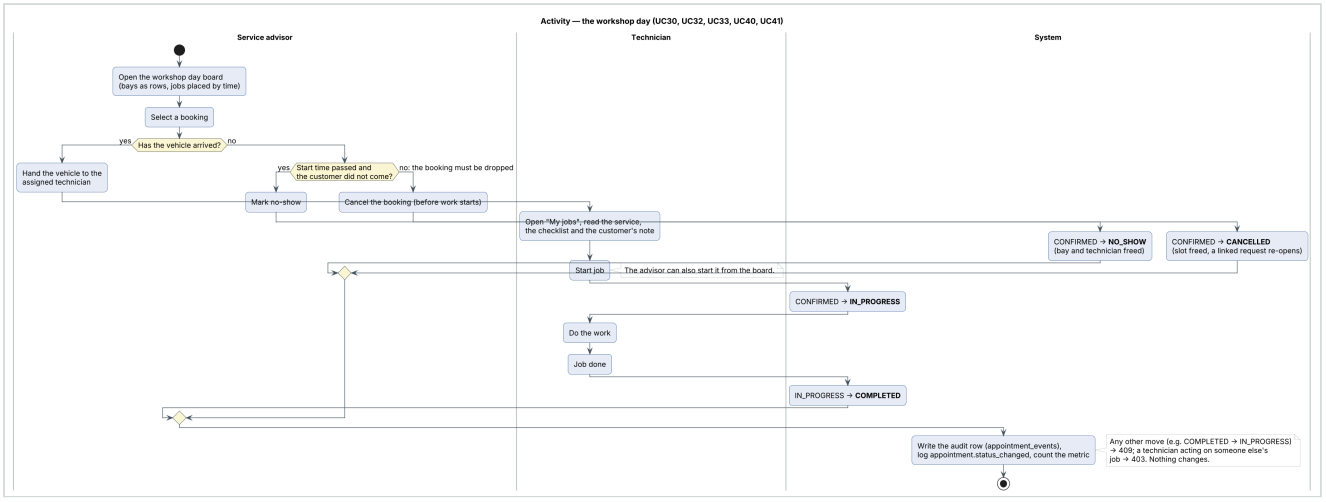
Use cases, staff



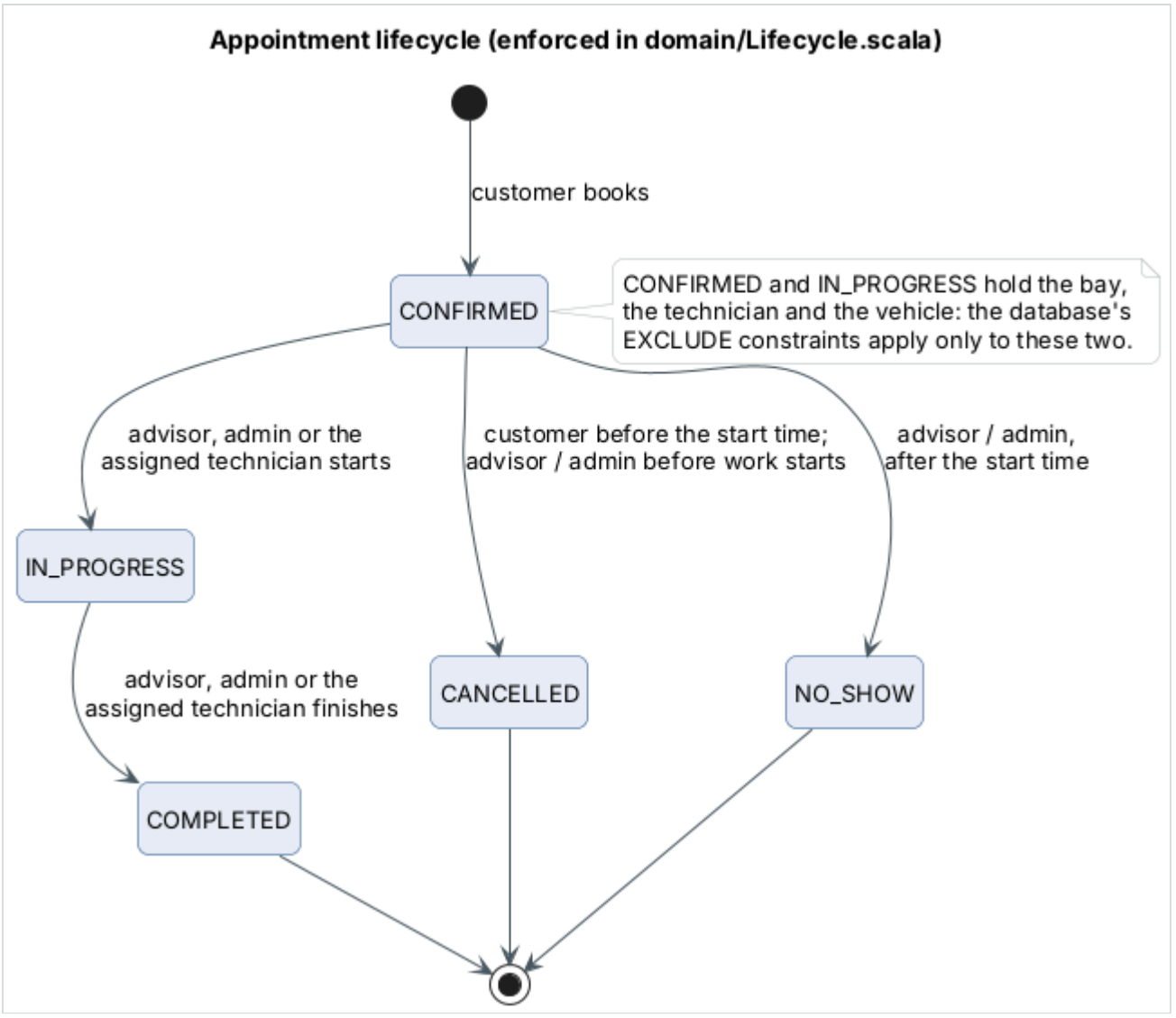
Activity, book a maintenance package



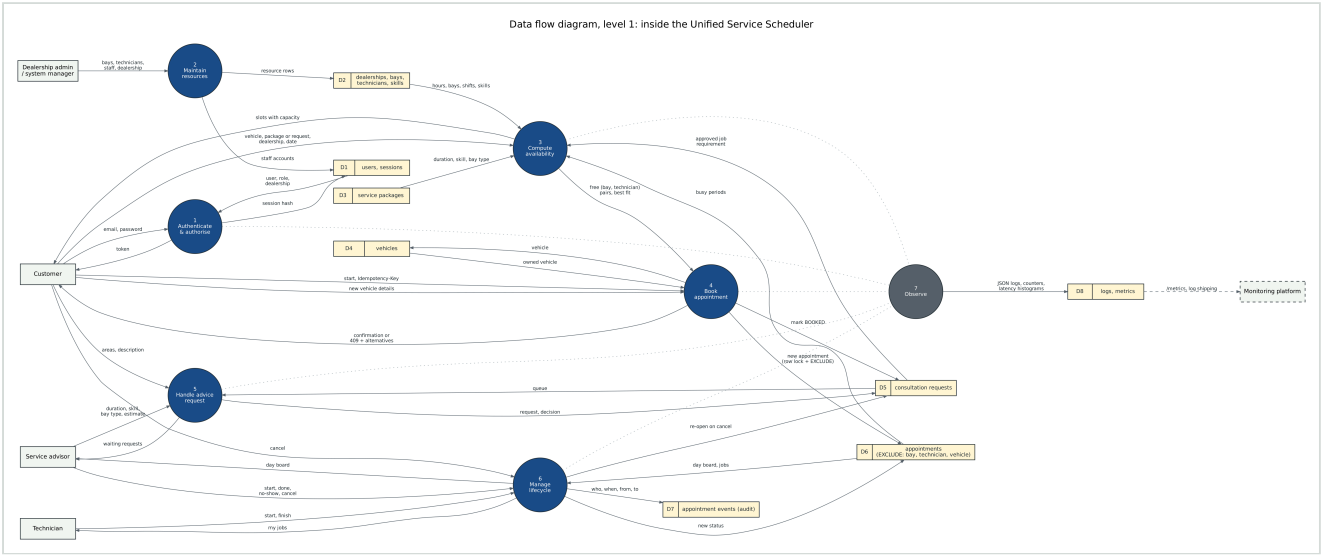
Activity, inspection or customisation



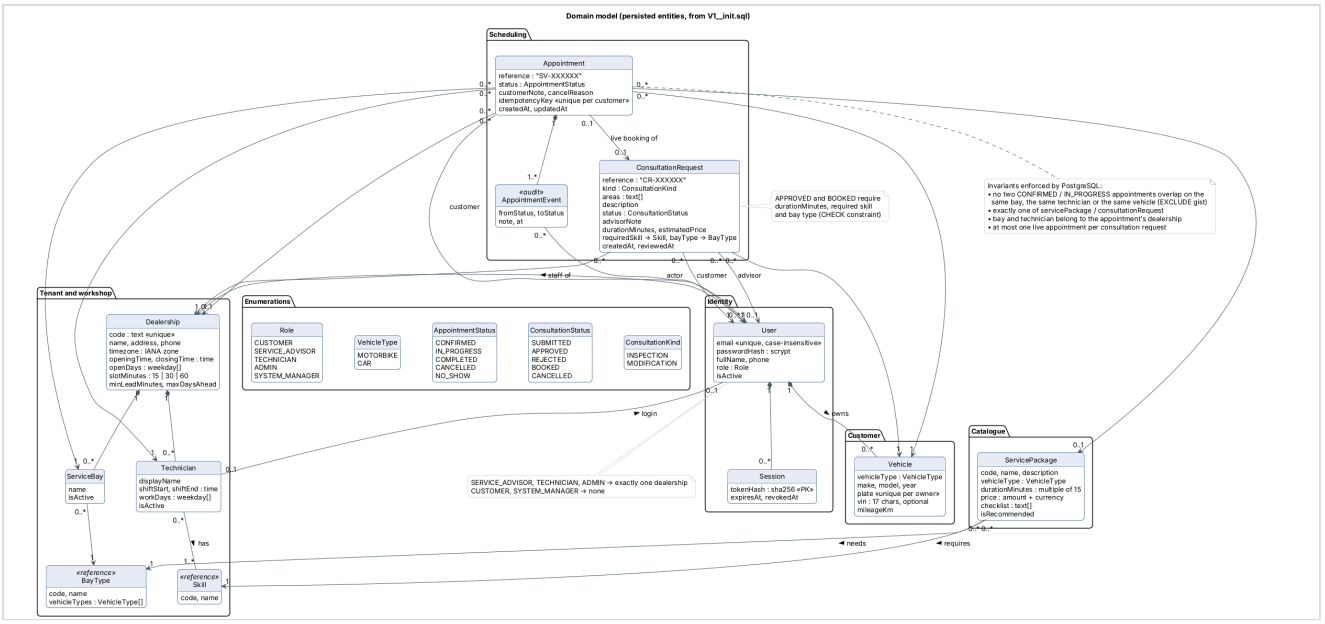
Activity, the workshop day



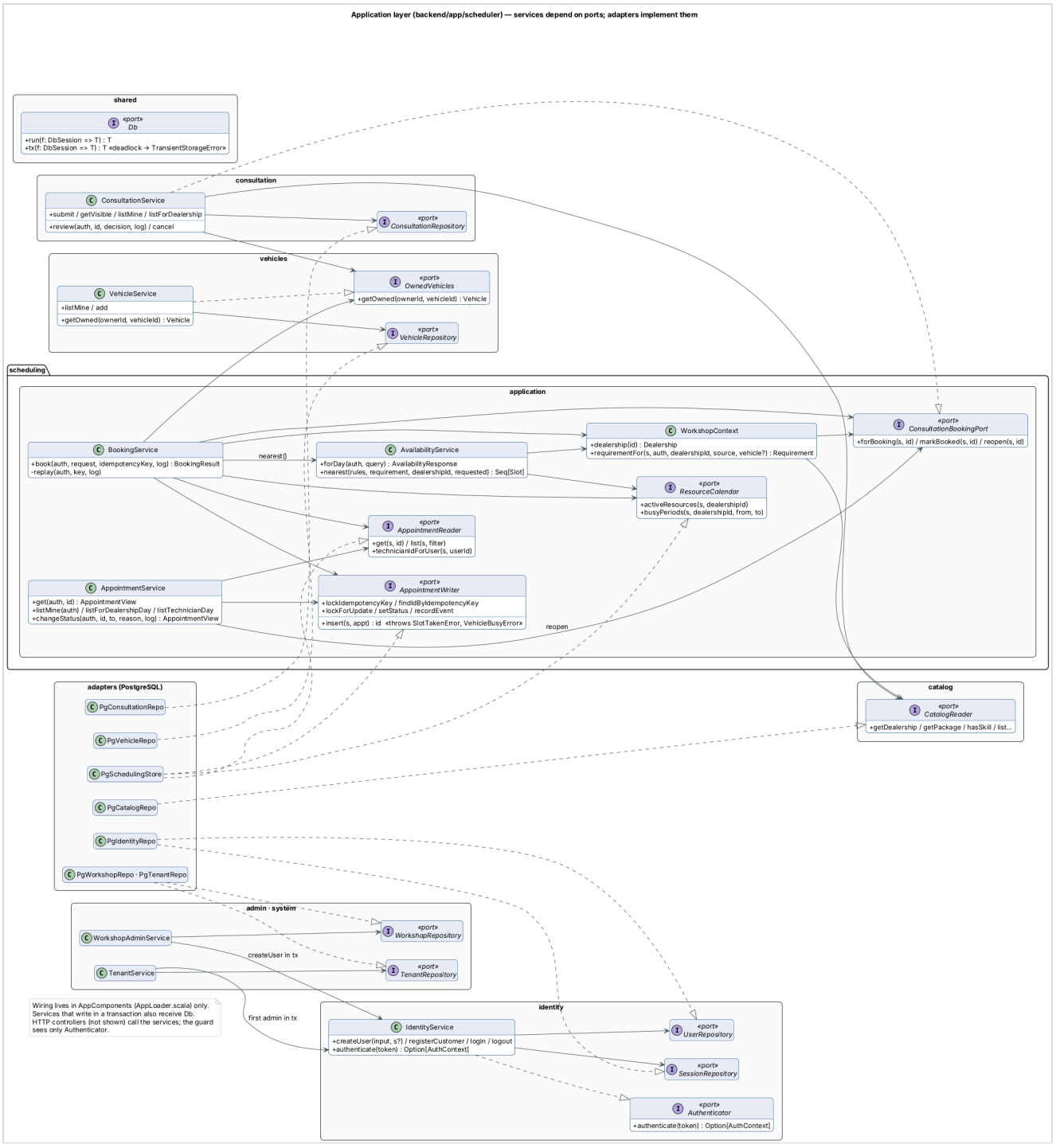
State, appointment lifecycle



Data flow, level 1



Class, domain model



Class, application layer

10. Technologies

Layer	Technology	Version	Why
Language	Scala	3.3.6 LTS (also 3.9.0, 3.10.0)	Strong types and enums for the domain
Runtime	Java (JDK)	21+	Virtual threads keep blocking JDBC code simple under load

Layer	Technology	Version	Why
Web framework	Play Framework (Pekko HTTP)	3.0.12	Routing, JSON, non-blocking HTTP server; compile-time wiring
Build	sbt	1.10.7	Standard Scala build; the launcher downloads the right version
Database	PostgreSQL	15+ (17 embedded)	Range types and exclusion constraints enforce BR-01 in the database
Migrations	Flyway	11.20.3	Versioned, locked migrations
SQL access	Anorm + HikariCP	2.7.0 + 6.3.0	Plain SQL with typed parameters; fast pool
Embedded database	Zonky embedded-postgres	2.1.0, PostgreSQL 17.6	`sbt run` and tests without Docker
JSON	play-json, Jackson	2.19.1 (pinned)	Validation with per-field errors
Logging	Logback + logstash-logback-encoder	8.0	One JSON object per log line
Metrics	Prometheus Java client	1.10.0	Standard `/metrics` endpoint
API docs	OpenAPI + Swagger UI	3.1 + 5.17.14	Contract served at `/docs`
Tests	ScalaTest (scalatestplus-play 7.0.2) + ScalaCheck	3.2.19 + 1.18.1	Readable specs; property-based tests for scheduling invariants
Frontend	React, React Router, TanStack Query	19, 7, 5	SPA with cached, auto-refreshing queries
Frontend build	Vite, TypeScript	8, 6	Fast dev server with API proxy
Dates	java.time (API), Luxon (UI)	—	Explicit time-zone arithmetic
Diagrams	PlantUML, Graphviz	1.2025.4	Diagrams as text, kept with the code

11. Installation guide

11.1 Prerequisites

Tool	Version	Check with
JDK	21 or newer (Temurin recommended)	`java -version`
sbt	1.10 or newer	`sbt --version`
Node.js	22 or newer (includes npm)	`node -v`
Git (optional)	any	`git --version`

Docker is **not** needed: the API starts its own PostgreSQL.

11.2 Install the tools

macOS: `brew install openjdk@21 sbt node`.

Linux (Debian / Ubuntu): install Temurin 21 and sbt from their official apt repositories, and Node.js 22 from NodeSource or `nvm`.

Windows with administrator rights: install the Temurin 21 MSI, the sbt MSI and the Node.js LTS MSI, then open a new terminal.

Windows without administrator rights (for example a school or company PC):

1. Download the **Temurin JDK 21 .zip** (not the MSI) from adoptium.net and unzip it to `C:\Users\<you>\tools\jdk-21`.
2. Download **sbt-1.10.x.zip** from scala-sbt.org and unzip it to `C:\Users\<you>\tools\sbt`.
3. Download the **Node.js .zip** for Windows and unzip it to `C:\Users\<you>\tools\node`.
4. Add them to *your user* PATH in PowerShell (no admin needed), then open a new terminal:

```powershell
$t = "$env:USERPROFILE\tools"
$p = [Environment]::GetEnvironmentVariable("Path", "User")
```

5. Check: `java -version`, `sbt --version`, `node -v`.

11.3 Get the code and run it

```
# unzip service-scheduler.zip (or git clone), then from the project root:
npm run setup     # once: installs the web app's dependencies
npm run dev       # starts the API (sbt run + embedded PostgreSQL) on :8080 and the web app
on :5173
```

`npm run dev` checks that `java` and `sbt` are on the PATH and that `npm run setup` has been run, and says what is missing. The first start downloads dependencies and compiles the Scala code; it takes one to three minutes. It is ready when the log shows `server.ready`.

Open	For
http://localhost:5173	The web app (demo accounts on the sign-in page, password `Demo@2026`)
http://localhost:8080/docs	Swagger UI
http://localhost:8080/health	Health check

To run only the API: `cd backend && sbt run`.

11.4 Common tasks

Task	How
Start with fresh demo data	Stop the app, delete `.data/pg-54330`, start again
Run the tests	`npm test` (95 tests), `npm run test:load` (1000-request bursts)

Task	How
Use another API port, for example 8081	Either run `cd backend && sbt "run 8081"`, or change `PlayKeys.playDefaultPort` in `backend/build.sbt` and `server.http.port` in `backend/conf/application.conf`. In both cases also change the proxy targets in `frontend/vite.config.ts` to `http://localhost:8081`
Use another web app port	Change `server.port` in `frontend/vite.config.ts`, and add the new origin to `CORS_ORIGIN` if the browser calls the API directly
Use your own PostgreSQL	`docker compose up -d` (or any PostgreSQL 15+), then set `DATABASE_URL=postgres://scheduler:scheduler@localhost:5432/scheduler` before `npm run dev`
Readable or JSON logs	`LOG_FORMAT=pretty` (default in `npm run dev`) or `LOG_FORMAT=json`
Use Scala 3.9 or 3.10	Change `scalaVersion` in `backend/build.sbt`; sbt downloads the compiler

11.5 Production-style package

```
cd backend
sbt stage
DATABASE_URL=postgres://user:pass@db-host:5432/scheduler \
APPLICATION_SECRET=<a long random string> \
CORS_ORIGIN=https://booking.example.com \
SEED_DEMO_DATA=false \
target/universal/stage/bin/service-scheduler
```

Build the web app with `npm --prefix frontend run build` and serve `frontend/dist` from any static web server that forwards `/api` to the API. Run several API instances behind a load balancer if needed (NFR-03); Flyway locks migrations so two instances never migrate at once.

11.6 Configuration

Variable	Default	Meaning
`DATABASE_URL`	empty → embedded PostgreSQL	`postgres://…` or `jdbc:postgresql://…`
`EMBEDDED_PG_PORT`	54330	Port of the embedded PostgreSQL
`DATA_DIR`	`../.data` (project root)	Where the embedded PostgreSQL keeps its files
`PORT`	8080	HTTP port of the packaged server
`CORS_ORIGIN`	`http://localhost:5173`	Comma-separated allowed browser origins
`SEED_DEMO_DATA`	true	Load demo data into an empty database
`LOG_FORMAT`, `LOG_LEVEL`	json, INFO	`pretty` for readable local lines
`SLOW_QUERY_MS`	200	Log queries slower than this
`SESSION_TTL_HOURS`	12	Sign-in lifetime
`DB_POOL_SIZE`	20	Connections per API instance

Variable	Default	Meaning
`APPLICATION_SECRET`	development value	Required by Play in production

11.7 Troubleshooting

Symptom	Cause	Fix
`'sbt' is not recognized` / `sbt: command not found`	sbt is not installed or not on the PATH	Section 11.2; open a new terminal after changing the PATH
`'vite' is not recognized`	The web app's dependencies are not installed	`npm run setup`
`spawn EINVAL` on Windows with Node 24	An old copy of the project (0.1.0)	Use version 0.2.0 or later; `package.json` shows the version
`Address already in use` / bind failure on 8080	Another program uses port 8080	Stop it, or move the API to another port (section 11.4)
The sign-in page says 404 or cannot reach the server	The API is not running or is still compiling	Wait for `server.ready`; check http://localhost:8080/health
Embedded PostgreSQL refuses to start on Linux as root	PostgreSQL never runs as root	Run as a normal user, or set `DATABASE_URL`
The first start is slow	sbt downloads dependencies and compiles	Only the first time; later starts take seconds

12. Out of scope and future work

Rescheduling (today: cancel and rebook), staff booking on behalf of a phone or walk-in customer, editing vehicles and technicians, managing packages, prices, breaks and holidays in the UI, a temporary hold while the customer confirms (HOLD → CONFIRM), parts ordering, notifications (SMS, Zalo, email), jobs needing several technicians, rate limiting on sign-in, OpenTelemetry tracing, a CI pipeline, the workshop-controller and parts-staff roles.